Guía del notebook `1_06_descenso_unpaso`: un paso de la regla delta

Puntos: 3 · **Tiempo:** 20 min · **Clase:** 4 · **Archivo:** `notebooks/1_06_descenso_unpaso.ipynb`
Teoría completa: Guía de la Clase 4, sección B2.

¿Qué vas a aprender?

- Los tres pasos con los que una neurona corrige sus pesos: **término de error**, **incremento** y **actualización**.
- A comprobar que, después de un paso, la neurona se equivoca **menos**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_06_descenso_unpaso.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos (regla delta)

Paso	Fórmula	En palabras
1. Término de error	$\delta = (y - \hat{y}) \cdot f'(h)$	Cuánto y hacia dónde corregir
2. Incremento	$\Delta w = \eta \cdot \delta \cdot x$ ($\Delta b = \eta \cdot \delta$)	Cuánto mover cada peso
3. Actualización	$w \leftarrow w + \Delta w$, $b \leftarrow b + \Delta b$	Mover los pesos

Con la sigmoide, $f'(h) = \sigma(h) \cdot (1 - \sigma(h))$. El notebook ya la trae como `sigmoid_prime`.

Paso a paso

Celda: `sigmoid`, `sigmoid_prime` y la clase `Neurona`

Ya están hechas. Ejecútala.

Celda `TODO (1 punto)`: `error_term(y, x, neurona)`

Pistas:

- h se obtiene con `neurona.combinacion_lineal(x)`;
- ŷ, con `neurona.forward(x)`;
- $f'(h)$, con `sigmoid_prime(h)`;
- multiplica $(y - \hat{y})$ por $f'(h)$.

Celda `TODO (1 punto)`: `incremento(eta, error_term, X)`

Devuelve η·δ·X.

Celda `TODO (1 punto)`: `actualizacion(neurona, incremento_W, incremento_b)`

Suma el incremento a `neurona.W`, y el del sesgo a `neurona.b`. Devuelve la neurona.

Celda "valores de ejemplo"

Usa tasa 2.0, x = [1, 1], y = 1, w = [0.1, 0.2] y b = 0.

Celda "Calcular la salida de la red"

Cambia `salida = None` por la predicción de la neurona, `neurona.forward(x)`. Lo demás ya llama a tus funciones.

✅ **Comprobación:**

Línea	Debe imprimir
Salida	≈ 0.5744
Término de error	≈ 0.1040
Incremento	≈ [0.2081 0.2081]
Incremento del sesgo	≈ 0.2081
Pesos actualizados	≈ [0.3081 0.4081]

Último punto (1 punto): ¿bajó la pérdida?

Agrega una celda que vuelva a predecir con la neurona actualizada y compare la pérdida ½(y − ŷ)² antes y después.

✅ **Comprobación:** la nueva salida es ≈ **0.7159**, más cerca de y = 1. La pérdida baja de **0.0905** a **0.0404**.

Errores frecuentes

Síntoma	Causa	Solución
Término de error ≈ −0.104	Escribiste (ŷ − y) en lugar de (y − ŷ)	El orden es **y − ŷ**
Incremento del sesgo con dos números	Pasaste `x` en lugar de `1`	Para el sesgo, la "entrada" es 1 (la celda ya lo hace)
`TypeError: ... NoneType`	Alguna función devuelve `None`	Completa las tres funciones y vuelve a ejecutarlas
Los pesos no cambian	`actualizacion` no guarda en `neurona.W`	Asigna `neurona.W = neurona.W + incremento_W`